ShipSmart — System Documentation & Interview Study Guide

A deep, codebase-grounded learning guide for Senior Backend / AI-Integrated Systems interviews

Covers: Polyglot architecture · RAG pipeline · MCP/Tooling · Task-based LLM routing

Contents

1. System Overview (Big Picture)
2. End-to-End Request Flow
3. Spring Boot Backend (Deep Dive)
4. FastAPI AI Service (Deep Dive)
5. RAG Concepts — Interview Focused
6. MCP / Tooling Concepts
7. LLM Design Decisions
8. Key Interview Talking Points
9. Common Pitfalls + What We Did Right
10. 7-Day Study Plan
11. Simplified Revision Notes

1. System Overview (Big Picture)

1.1 What ShipSmart Is

ShipSmart is a shipping comparison and management platform. Users search for shipping quotes, save options, get AI-powered shipping advice, and track bookings. The product question that drives the architecture is: 'How do we combine reliable transactional logic (quotes, bookings) with AI-assisted guidance (advice, recommendations) without letting one compromise the other?'

1.2 Polyglot 3-Service Architecture

ShipSmart is a polyglot monorepo (Nx 22.3 + pnpm) containing three deployable services:

- Frontend (apps/web) — React 19 + Vite + TypeScript + shadcn/ui + Tailwind. Static site on Render.
- Java API (apps/api-java) — Spring Boot 4.0.5 (Java 25). Owns transactional data: quotes, saved options, booking redirects. Supabase PostgreSQL + JWT auth.
- Python API (apps/api-python) — FastAPI 0.135.3 (Python 3.13). Owns AI features: RAG pipeline, LLM integration, tool orchestration, advisor endpoints, recommendations.

Supporting infrastructure: Supabase (Postgres + Auth) and a small set of legacy edge functions kept feature-flagged as a fallback.

1.3 High-Level Architecture Diagram

```
                    ┌──────────────────────────┐
                    │  React 19 Frontend       │
                    │  (apps/web)              │
                    │  - Quote form            │
                    │  - Saved options         │
                    │  - Advisor page          │
                    └──────────────────────────┘
                                │
                                │  HTTPS / JSON
                    ┌───────────┴──────────────┐
                    │                          │
          ┌─────────▼─────────┐        ┌─────────▼─────────┐
          │  Java API         │        │  Python API       │
          │  Spring Boot      │        │  FastAPI          │
          │  apps/api-java    │        │  apps/api-python  │
          │                   │        │                   │
          │  Owns:            │        │  Owns:            │
          │  • Quotes         │        │  • RAG pipeline   │
          │  • Saved options  │        │  • Tool registry  │
          │  • Bookings       │        │  • Advisors       │
          │  • JWT auth       │        │  • LLM router     │
          └───────────────────┘        └─────────┬─────────┘
                    │                        │         │
          ┌─────────▼─────────┐        ┌─────▼─────┐   ┌──▼──────────┐
          │   Supabase        │        │ Carrier APIs│  │  LLM APIs   │
          │                   │        │           │   │             │
```

```
│   Postgres+Auth   │                │ UPS/FedEx/   │   │  OpenAI      │
└────────────────────┘                │ DHL/USPS     │   │  Gemini      │
                                      │ (or Mock)    │   │  (Echo fb)   │
                                      └──────────────┘   └──────────────┘
```

Important: the Java and Python APIs do not call each other. The frontend orchestrates between them. This is intentional — each service is independently deployable and the contract surface between them is the frontend's data shape, not an internal RPC.

1.4 Why Split Java and Python?

The split is driven by ownership, not language preference:

- Spring Boot is the system of record. It must be transactional, validated, auditable, and JWT-authenticated. Java/Spring is a mature, boring choice that nails this.
- FastAPI hosts AI workloads where the ecosystem (LangChain-style libraries, OpenAI/Gemini SDKs, embeddings, vector libs) is Python-first. Mixing async I/O for LLM and HTTP calls is also natural in FastAPI.
- Failure isolation: a flaky LLM provider or a poisoned RAG document cannot bring down the quoting/booking flow. If the Python API is down, the frontend simply hides the advisor card.
- Independent deploy and scale: the AI service is more volatile (model swaps, prompt tweaks). Keeping it on its own deploy cycle protects the transactional service.

2. End-to-End Request Flow

2.1 Scenario: 'Best shipping option for a fragile item'

Concrete walk-through of what happens, layer by layer, when the user types this question into the Advisor page.

```
┌──────────────┐   POST /api/v1/advisor/shipping
│  Frontend    │ ──────────────────────────────►
│ AdvisorPage  │      { query, context }
└──────────────┘
                                  │
                                  ▼
                  ┌──────────────────────────────┐
                  │ FastAPI route: advisor.shipping │
                  │ apps/api-python/app/api/routes/ │
                  │ advisor.py:32                   │
                  └──────────────────────────────┘
                                  │ pull rag + tool_registry +
                                  │ llm_router from app.state
                                  ▼
                  ┌──────────────────────────────┐
                  │ get_shipping_advice()           │
                  │ services/shipping_advisor_service │
                  └──────────────────────────────┘
                                  │
                                  │
        ┌─────────────────────────┼──────────────────────────┐
        ▼                         ▼                          ▼
┌──────────────┐          ┌──────────────┐          ┌──────────────┐
│ RAG retrieve │          │ Tool execution │          │ LLM reasoning │
│ embed query →│          │ (only when     │          │ via router    │
│ vector search│          │ context flags  │          │ for_task(     │
│ top_k chunks │          │ trigger them)  │          │  REASONING)   │
└──────────────┘          └──────────────┘          └──────────────┘
        │                         │                          │
        └─────────────────────────┼──────────────────────────┘
                                  ▼
                  ┌──────────────────────────────┐
                  │ build_advisor_prompt( query,    │
                  │   context = retrieved + tools ) │
                  └──────────────────────────────┘
                                  ▼
                  ┌──────────────────────────────┐
                  │ OpenAI Chat Completions         │
                  │ (reasoning task → openai)       │
                  └──────────────────────────────┘
                                  ▼
                  ┌──────────────────────────────┐
                  │ ShippingAdvisorResponse JSON    │
                  │ { answer, reasoning_summary,    │
                  │   tools_used, sources,          │
                  │   context_used }                │
```

```
└──────────────────┬──────────────────┘
                   ▼
          ┌──────────────────┐
          │  Frontend renders  │
          │  in AdvisorCard    │
          └──────────────────┘
```

2.2 Step-by-Step Narration

1. User types the query and hits submit on AdvisorPage. The frontend POSTs to /api/v1/advisor/shipping on the Python API. The Java API is not involved — this is a pure AI flow.
2. FastAPI route handler (advisor.py) pulls three things off app.state: rag (embedding_provider, vector_store), tool_registry, and llm_router. These were built once at startup by the lifespan hook in main.py.
3. The route asks the router for the reasoning client: llm_router.for_task(TASK_REASONING). With the recommended config that returns the OpenAI client.
4. get_shipping_advice() runs the RAG retrieval: embed the query, cosine-search the in-memory vector store, return the top-K chunks with their source paths and scores.
5. If the request context contains hints (e.g. an address dict), the service runs ValidateAddressTool / GetQuotePreviewTool from the registry. Tool outputs are appended to the prompt context.
6. build_advisor_prompt() composes a single prompt with the original query, the retrieved chunks, and the tool outputs. The prompt is grounded — the system message says 'answer ONLY from the provided context'.
7. The OpenAI client sends the messages to gpt-4o-mini with configured timeout, max_tokens, and temperature. On any error, AppError(502) is raised by the client and the global error handler converts it to a clean JSON response.
8. The service builds a structured response: answer text, reasoning summary, tools_used list, sources list (file path + chunk index + score), and context_used flag.
9. The frontend renders the structured fields. Sources become citation chips so the user can see exactly which document the answer came from.

2.3 Contrast: RAG-Only Query Flow

POST /api/v1/rag/query is simpler — no tool layer. The route calls llm_router.for_task(TASK_SYNTHESIS) which returns the Gemini client. Retrieval → prompt → Gemini → return answer + sources. Same retrieval code, different LLM, different prompt (build_rag_prompt vs build_advisor_prompt).

3. Spring Boot Backend (Deep Dive)

3.1 Role

Spring Boot is the system of record for everything money-shaped or audit-shaped: the user's quote requests, their saved options, and their booking redirects. Anything that has to be correct even if the AI service is on fire lives here.

3.2 Key Endpoints

- POST /api/v1/quotes — submit a 3-step quote form, get carrier comparison.
- CRUD /api/v1/saved-options — JWT-protected, per-user saved quotes.
- POST /api/v1/bookings/redirect — record that the user clicked through to a carrier site to book.

3.3 Why Java for the Transactional Layer?

- Mature transactional story: Spring Data JPA, declarative @Transactional, validated DTOs, global exception handlers.
- Strong type system catches schema/contract drift early — important for the data the business cares about.
- JWT auth with Spring Security is well-trodden ground; tokens come from Supabase Auth and are validated server-side.
- It is the boring choice. The right amount of boring for the part of the system that must never lose data.

3.4 How Java Interacts With Python

It does not. By design. The frontend calls each service directly and merges results in the UI. The two backends share nothing except the user identity (Supabase JWT). This means:

- No circular dependency between services.
- Independent deploys and rollbacks.
- An LLM outage cannot block a quote being saved.
- Each backend has exactly one persistence story.

4. FastAPI AI Service (Deep Dive)

4.1 Core Architecture

The Python service is built around a single principle: build everything heavy once at startup, hang it off app.state, and let route handlers borrow it. This makes request handling thin and testable.

Lifecycle (apps/api-python/app/main.py)

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    # 1. Shared httpx client
    app.state.http_client = httpx.AsyncClient(...)

    # 2. RAG components
    embedding_provider = create_embedding_provider()
    vector_store       = create_vector_store()

    # 3. LLM router (one client per task)
    llm_router = create_llm_router()
    app.state.llm_router = llm_router

    # 4. Back-compat shim — synthesis client also under app.state.rag
    app.state.rag = {
        "embedding_provider": embedding_provider,
        "vector_store":       vector_store,
        "llm_client":         llm_router.for_task(TASK_SYNTHESIS),
    }

    # 5. Tool registry bound to a shipping provider
    shipping_provider = create_shipping_provider()
    tool_registry = ToolRegistry()
    tool_registry.register(ValidateAddressTool(shipping_provider))
    tool_registry.register(GetQuotePreviewTool(shipping_provider))
    app.state.tool_registry = tool_registry

    yield

    await app.state.http_client.aclose()
```

Config System

app/core/config.py uses pydantic-settings. Every env var has a typed field with a default, so the app boots even with an empty .env. Properties like cors_origins_list and is_production keep the route handlers free of string-parsing.

4.2 RAG Pipeline

Folder layout of knowledge base

```
apps/api-python/data/documents/
├── carriers/   (UPS, FedEx, DHL, USPS overviews)
├── guides/     (FAQ, ground vs express, packaging, address quality)
├── policies/   (returns/claims, international, comparisons)
└── scenarios/  (recommendation tradeoffs, delays/exceptions)
```

Ingestion (app/rag/ingestion.py)

10. load_documents(directory) walks **/*.txt and **/*.md recursively. The relative path becomes the source name (e.g. carriers/ups-overview.md), so retrieval results are self-locating.
11. chunk_text(content, chunk_size=500, chunk_overlap=50) splits each document into overlapping windows with their source + index.
12. embedding_provider.embed(texts) turns chunk text into vectors. With EMBEDDING_PROVIDER='openai' it uses text-embedding-3-small; otherwise LocalHashEmbedding (deterministic, no semantics — fine for dev).
13. vector_store.add(stored_chunks) persists into InMemoryVectorStore. zip(strict=True) catches any chunk/embedding count mismatch loudly.

Retrieval

rag_query() embeds the user query, calls vector_store.search() (cosine similarity, top_k from settings.rag_top_k), then passes the chunks to build_rag_prompt() and the LLM. The response carries the sources back to the caller so the UI can cite them.

4.3 MCP / Tooling Layer

Tools are deterministic, schema-validated functions that the LLM (or our own service code) can call. They exist so that anything with a right answer — address validation, quote lookup — is computed by code, not hallucinated by the model.

```
class Tool(ABC):
    name: str
    description: str
    parameters: list[ParameterSchema]
    async def execute(self, **kwargs) -> ToolResult: ...
    def schema(self) -> dict: ...
    def validate_input(self, kwargs) -> None: ...

class ToolRegistry:
    register(tool)
    get(name) -> Tool
    list_tools() -> list[Tool]
    list_schemas() -> list[dict]   # for /orchestration/tools
```

ValidateAddressTool and GetQuotePreviewTool both delegate to the configured ShippingProvider, so tools work with mock or real carriers without code changes.

4.4 Provider Integration

ShippingProvider is an ABC with validate_address() and get_quote_preview(). create_shipping_provider() reads SHIPPING_PROVIDER from config, dynamically imports the matching module, validates required credentials, and falls back to MockShippingProvider on any failure. The app never crashes because of carrier env-var problems.

```
SHIPPING_PROVIDER=ups   →  UPSProvider   (needs UPS_CLIENT_ID + SECRET)
SHIPPING_PROVIDER=fedex →  FedExProvider (needs FEDEX_CLIENT_ID + SECRET)
SHIPPING_PROVIDER=dhl   →  DHLProvider   (needs DHL_API_KEY + SECRET)
SHIPPING_PROVIDER=usps  →  USPSProvider  (needs USPS_CLIENT_ID + SECRET)
SHIPPING_PROVIDER=mock  →  MockShippingProvider
unset / unknown / bad creds → MockShippingProvider (with warning)
```

4.5 LLM Integration

LLMClient is an ABC with one method: complete(messages) -> str. Concrete implementations:

- OpenAIClient — AsyncOpenAI SDK, configurable model/timeout/temperature/max_tokens. Maps SDK errors to AppError(502).
- GeminiClient — REST via httpx. Includes a _messages_to_gemini_contents helper that merges the OpenAI-style system message into the first user turn (Gemini has no system role).
- LlamaClient — talks to a local Ollama server using the OpenAI-compatible /v1 endpoint. Useful for offline/dev.
- EchoClient — placeholder. Returns retrieved context as text. Used as the universal safety net.

4.6 Task-Based LLM Routing (the Phase 14 work)

The router (app/llm/router.py) maps a logical task to a concrete LLMClient. Three task names exist:

- TASK_REASONING — used by /advisor/shipping and /advisor/tracking. Reasoning over RAG context + tool results, where instruction-following matters more than cost.
- TASK_SYNTHESIS — used by /rag/query and /advisor/recommendation. Grounded text generation from already-structured input. Cheaper, faster models are fine here.
- TASK_FALLBACK — the safety net. Always points at something that always works (Echo by default).

Resolution order per task

14. LLM_PROVIDER_<TASK> if set (e.g. LLM_PROVIDER_REASONING=openai)
15. Legacy LLM_PROVIDER (so older deployments keep working)
16. LLM_PROVIDER_FALLBACK (the configured fallback provider)

17. EchoClient (always available)

```
# Recommended production config
LLM_PROVIDER_REASONING=openai   OPENAI_API_KEY=sk-...
LLM_PROVIDER_SYNTHESIS=gemini   GEMINI_API_KEY=...
LLM_PROVIDER_FALLBACK=echo

# Resulting router on startup:
# LLM router initialized: {
#   'reasoning': 'openai',
#   'synthesis': 'gemini',
#   'fallback': 'echo'
# }
```

Crucially, each task degrades independently. A missing OpenAI key only collapses the reasoning task to echo — synthesis with Gemini keeps working. This is verified by test_missing_key_falls_back_to_fallback_provider.

5. RAG Concepts (Interview-Focused)

5.1 What is RAG?

Retrieval-Augmented Generation. Instead of asking the LLM to answer from its frozen training data, you (1) retrieve relevant passages from your own knowledge base, (2) put them in the prompt, (3) instruct the model to answer ONLY from those passages. The model becomes a reading-comprehension engine over your data, not a memory engine.

5.2 Why Not Just Use the LLM Alone?

- Hallucination: LLMs invent confident answers when they don't know.
- Stale knowledge: training cutoffs are months or years old.
- No private data: an LLM has never seen your carrier policies.
- No citations: free-form generation can't tell users where the answer came from.

RAG fixes all four with one design move: ground every answer in retrieved text and surface that text as a citation.

5.3 Chunking

Why not embed whole documents? Two reasons. (1) The model has a context limit. (2) Long documents dilute the embedding — a single vector can't represent twenty different topics. We chunk into ~500-character windows with 50-character overlap. The overlap means a sentence that straddles a boundary still appears in at least one chunk in full.

In this codebase: app/rag/chunking.py + RAG_CHUNK_SIZE / RAG_CHUNK_OVERLAP env vars.

5.4 Embeddings

An embedding turns text into a fixed-length vector such that semantically similar text lands at nearby points. 'How long does ground shipping take?' and 'UPS Ground delivery time' end up close together even though they share almost no words.

- Production: OpenAIEmbedding with text-embedding-3-small (256-dim).
- Dev/CI: LocalHashEmbedding — deterministic hash-based vectors. No real semantics; lets the pipeline run without external calls.

5.5 Vector Search

Cosine similarity between the query embedding and every stored chunk embedding. Top-K (default 3) wins. In production you'd use a real ANN index (pgvector, Qdrant, FAISS); we use InMemoryVectorStore with a brute-force scan because the corpus is small (~150 chunks) and the goal is to demonstrate the shape of the system, not the index.

5.6 The Retrieve-Then-Read Flow

```
query
  |
  ▼
```

```
embed(query) ──────►  vector
                        │
                        ▼
              vector_store.search(vector, top_k=3)
                        │
                        ▼
            top_k chunks (text + source + score)
                        │
                        ▼
       build_rag_prompt(query, [chunk1, chunk2, chunk3])
                        │
                        ▼
              LLM.complete(messages)
                        │
                        ▼
                grounded answer + sources
```

6. MCP / Tooling Concepts

6.1 Why LLMs Need Tools

An LLM is a text predictor. Ask it 'is 90210 a valid ZIP code in California?' and you'll often get a confident yes — even for ZIPs that don't exist. Ask it 'what does UPS Ground cost from Boston to Seattle for a 5lb package?' and it will invent a number. Tools fix this by routing those questions to code.

6.2 Free Text vs Structured Execution

```
                    ┌──────────────────────────────────┐
                    │  LLM (text in, text out)         │
                    └──────────────────────────────────┘
                          │
                          │
      ┌───────────────────┼────────────────────┐
      ▼                   ▼                    ▼
   free text          structured tool      structured tool
   generation         call (validated)     call (validated)
      │                   │                    │
      ▼                   ▼                    ▼
   '1-5 days'          ValidateAddress      GetQuotePreview
   'usually fine'      → real provider      → real provider
                       → real result        → real result
```

The free-text path is fine for tone and reasoning. The structured path is where ground truth lives. A good system uses both, with clear handoffs.

6.3 How Our Tool Layer Works

18. Each tool subclasses Tool with name, description, parameters, and async execute().
19. ToolRegistry holds them. /api/v1/orchestration/tools returns the JSON schemas — that is the MCP-style discovery surface.
20. ValidateAddressTool and GetQuotePreviewTool delegate to the active ShippingProvider, which is mock or real depending on env vars.
21. Inputs are validated before execute() runs. Errors raise AppError so the global handler maps them to clean HTTP responses.
22. Tool outputs are appended to the LLM prompt as evidence so the model's answer is grounded in real data, not invented.

6.4 How This Avoids Hallucination

- Address questions go to ValidateAddressTool, not to the LLM.
- Pricing/ETA questions go to GetQuotePreviewTool, not to the LLM.
- Open-ended advice goes to the LLM, but with retrieved policy text in the prompt and an instruction to answer only from that text.
- Sources are returned to the UI so users can verify any answer.

7. LLM Design Decisions

7.1 Why Multiple LLMs?

Different tasks have different cost/quality curves. Reasoning over tool results and policy snippets benefits from a stronger instruction-following model. Synthesizing a short summary from already-structured data is a much simpler job and a cheaper model handles it indistinguishably. Mixing providers also removes single-vendor risk.

7.2 Why Task-Based Routing (Not a Single Model)?

- Cost: Gemini Flash is roughly an order of magnitude cheaper than OpenAI per token. Sending light synthesis to it cuts the LLM bill without hurting users.
- Failure isolation: a Gemini outage breaks RAG q&a but the advisor reasoning path stays up.
- Operability: ops can swap a task's provider via env var only. Zero code change, zero new endpoint.
- Honest interview answer: it's also a clean abstraction to talk about — task → provider, with explicit fallback. Easy to whiteboard.

7.3 Why OpenAI for Reasoning?

- Strong instruction-following on grounded prompts.
- Reliable JSON / structured output behaviour.
- gpt-4o-mini is the cheapest model in the family that handles our prompts reliably; we don't need 4o for this workload.

7.4 Why Gemini for Synthesis?

- Roughly 10× cheaper per token than OpenAI's small models.
- Quality difference on grounded summarisation is small to nil.
- Latency on Flash is noticeably lower for short outputs.

7.5 Why a Fallback at All?

EchoClient never makes a network call and never fails. Wiring it as the safety net guarantees the API stays up — and interviews can demo end-to-end — when keys are missing or both providers are down.

7.6 Tradeoffs

- Two providers means two SDK surfaces, two error modes, two prompt quirks (Gemini has no system role). The router contains this complexity in one file.
- Per-task tuning (timeout/temperature/max_tokens) is shared across tasks today — that's a known limitation, easy to extend.
- Mixed providers can produce slightly different tone between reasoning and synthesis tasks. Acceptable here because each task talks to a different UI surface.

8. Key Interview Talking Points

8.1 'Explain your system architecture.'

ShipSmart is a polyglot monorepo with three deployable services. A React frontend orchestrates between two backends: a Spring Boot service that owns transactional data (quotes, saved options, bookings) backed by Supabase Postgres with JWT auth, and a FastAPI service that owns AI features — RAG, tools, advisors, and a task-based LLM router. The two backends never call each other; they share only the user's JWT identity. That gives independent deployability and means an LLM outage cannot block a quote being saved.

8.2 'How does your RAG pipeline work?'

On startup we walk data/documents recursively, chunk each file into ~500-char overlapping windows, embed them — OpenAI text-embedding-3-small in production, a deterministic hash embedding in dev — and store them in an in-memory vector store. At query time we embed the query, take cosine top-3, and call build_rag_prompt() which puts the chunks in a system-grounded prompt that tells the LLM to answer only from the context. The endpoint returns the answer plus a list of sources (file path + chunk index + similarity score) so the UI can cite them.

8.3 'How do you avoid hallucination?'

23. Grounding: the system prompt explicitly says 'answer ONLY from the provided context'.
24. Retrieval over generation: we put real document text into the prompt instead of trusting the model's parametric memory.
25. Tools for ground-truth questions: addresses and quotes go through ValidateAddressTool / GetQuotePreviewTool, not the LLM.
26. Citations: we return sources to the UI so the user can verify any answer.
27. Echo fallback: if the LLM is unavailable we return the retrieved context verbatim rather than nothing.

8.4 'Why separate Java and Python services?'

Ownership and failure isolation, not language preference. Spring is the boring, mature choice for transactional data — JPA, @Transactional, validated DTOs, JWT auth. FastAPI is where the AI ecosystem lives — OpenAI/Gemini SDKs, async I/O, embedding libraries. Splitting them means the AI service can deploy faster, fail independently, and never block the quoting flow.

8.5 'How would you scale this system?'

- Replace InMemoryVectorStore with pgvector or Qdrant — same VectorStore ABC, swap one factory.
- Add a real ANN index — brute force is fine at 150 chunks but breaks down past ~10k.
- Cache embeddings of frequent queries (Redis) and cache LLM completions for identical prompts.
- Move ingestion offline — currently it's a route; in production it would be a background job.
- Add per-task timeouts and circuit breakers around each LLM provider.

- Horizontal scale FastAPI behind a load balancer; the service is stateless apart from app.state caches that are easy to externalise.

8.6 'How would you improve this system?'

- Per-task LLM tuning (LLM_TEMPERATURE_SYNTHESIS, LLM_MODEL_REASONING).
- Persistent vector store with metadata filtering.
- Function calling / proper MCP server interface so external clients can discover and call our tools.
- Multi-turn advisor conversations with history.
- Rate limiting and auth on the Python API.
- Eval harness for RAG quality (recall@k, faithfulness scoring).

9. Common Pitfalls + What We Did Right

9.1 Pitfalls People Hit Building RAG

- Embedding whole documents — breaks retrieval quality. Fix: chunk with overlap.
- Letting the LLM answer from memory when context is empty — leads to hallucination. Fix: prompt explicitly says 'no context → say so'.
- No citations — users can't verify the answer. Fix: pass sources back from retrieval to API response.
- Single global LLM client — one outage takes everything down. Fix: per-task router with independent fallback.
- Hardcoded provider — no way to swap models without code change. Fix: env-driven factory + ABC.
- Tools that re-implement business logic — drift between code and what the LLM 'knows'. Fix: tools delegate to the same provider abstraction the rest of the system uses.
- Crashing the app on missing env vars — fragile. Fix: every factory degrades to a safe default with a warning.

9.2 What This Codebase Does Right

- Clean ABCs (LLMClient, EmbeddingProvider, VectorStore, ShippingProvider, Tool) — implementations are interchangeable.
- Single startup, hang-on-app.state pattern — request handlers are thin and testable.
- Task-based routing built on a config-driven resolver — no heuristics, no hidden behaviour.
- Per-task fallback so partial outages stay partial.
- Recursive document loader using the relative path as the source name — citations are self-locating.
- 171 Python tests including the router/factory/missing-key cases. The code is pinned by tests, not vibes.

10. 7-Day Study Plan

Each day pairs reading with a hands-on action and an out-loud explanation. Speaking the answer is what locks it in for the interview.

Day 1 — System Overview & Architecture

- Read: docs/current-system-state.md, this guide §1 and §2.
- Run: start both backends locally; hit /health on each.
- Understand: the three services and the rule that Java and Python don't talk to each other.
- Out loud: explain the architecture in 90 seconds without looking at the diagram.

Day 2 — Spring Boot Layer

- Read: apps/api-java controllers, services, and security config.
- Run: POST a quote, save an option, hit the booking redirect endpoint.
- Understand: why transactional data lives here, not in Python.
- Out loud: explain why splitting Java and Python is a feature not a cost.

Day 3 — FastAPI Skeleton & Lifecycle

- Read: apps/api-python/app/main.py, app/core/config.py.
- Run: uv run uvicorn app.main:app --reload, watch the startup logs (LLM router init line).
- Understand: the lifespan() pattern and how app.state holds prebuilt clients.
- Out loud: explain why building things once at startup is better than per-request construction.

Day 4 — RAG Pipeline

- Read: app/rag/{ingestion,chunking,embeddings,vector_store}.py + app/services/rag_service.py.
- Run: POST /api/v1/rag/ingest, then POST /api/v1/rag/query with a real question.
- Understand: ingest → chunk → embed → store → retrieve → prompt → generate.
- Out loud: explain RAG to someone who has never heard the term, in two minutes.

Day 5 — Tools / MCP Layer

- Read: app/tools/{base,registry,address_tools,quote_tools}.py + app/api/routes/orchestration.py.
- Run: GET /api/v1/orchestration/tools to see the schema list. POST /api/v1/orchestration/run with an explicit tool name.
- Understand: why deterministic tools sit alongside the LLM and how they avoid hallucination on factual questions.
- Out loud: explain the 'free text vs structured execution' split using ValidateAddressTool as the example.

Day 6 — LLM Integration & Task-Based Routing

- Read: app/llm/client.py, app/llm/router.py, app/llm/prompts.py.

- Run: try LLM_PROVIDER_REASONING=openai LLM_PROVIDER_SYNTHESIS=gemini and watch the router init log. Then break a key on purpose and confirm only one task degrades.
- Understand: per-task resolution order, the role of EchoClient, and why the abstraction sits at the ABC layer.
- Out loud: explain why a single LLM client would have been worse and walk through the resolution order from memory.

Day 7 — Provider Layer + Frontend Integration + Mock Interview

- Read: app/providers/{__init__,shipping_provider,mock_provider,ups_provider}.py + apps/web src AdvisorPage and RecommendationCard.
- Run: SHIPPING_PROVIDER=ups with empty creds → confirm fallback to mock; then SHIPPING_PROVIDER=mock and trace a full advisor request from the UI.
- Understand: how the same Tool / Service / LLM stack works against mock and real carriers.
- Out loud: do all six §8 talking points without notes. Time them.

11. Simplified Revision Notes

11.1 One-Page Mental Model

```
ShipSmart = 3 services:
  Frontend (React)  →  React 19, talks to BOTH backends
  Java API          →  transactional (quotes, saved, bookings)
  Python API        →  AI (RAG, tools, advisors, LLM router)

Java and Python NEVER call each other. Frontend orchestrates.

Python API has 5 layers:
  config       → pydantic-settings, env-driven
  providers    → ShippingProvider ABC, mock + 4 carriers
  rag          → ingest → chunk → embed → store → retrieve
  tools        → ToolRegistry, deterministic, schema-validated
  llm          → LLMClient ABC + LLMRouter (task → client)

Task-based routing:
  reasoning  → OpenAI (advisor endpoints)
  synthesis  → Gemini (rag/query, recommendation)
  fallback   → Echo (always works)

Resolution: LLM_PROVIDER_<TASK> → LLM_PROVIDER → fallback → Echo
```

11.2 30-Second Pitch

ShipSmart is a shipping comparison platform. It splits transactional logic (Spring Boot, Postgres, JWT) from AI features (FastAPI with a RAG pipeline, a tool registry, and a task-based LLM router that points reasoning at OpenAI and synthesis at Gemini, with an Echo fallback so the system never fails on missing keys). The two backends never call each other — the frontend orchestrates — which gives independent deployability and failure isolation.

11.3 Quick-Reference Table

Concept	Where in code	One-line summary
Lifecycle wiring	app/main.py	Build everything once at startup; hang on app.state.
Config	app/core/config.py	pydantic-settings, every field has a default.
RAG ingestion	app/rag/ingestion.py	Recursive walk → chunk → embed → store.
RAG query	app/services/rag_service.py	Embed query → top-K → grounded prompt → LLM.
Tool registry	app/tools/registry.py	Discoverable, schema-validated, deterministic.
Provider factory	app/providers/__init__.py	Env-driven, falls back to mock on bad creds.

LLM client ABC	app/llm/client.py	complete(messages) → str. Four implementations.
LLM router	app/llm/router.py	Task → client, resolved at startup, per-task fallback.
Reasoning task	advisor.py routes	OpenAI by default. Used by /advisor/shipping & /tracking.
Synthesis task	rag.py + advisor.py	Gemini by default. Used by /rag/query & /recommendation.

11.4 Final Sanity Checks Before the Interview

- Can I draw the architecture in under 60 seconds without looking?
- Can I explain RAG without using the word 'RAG'?
- Can I name the four LLMClient implementations and one reason each exists?
- Can I walk the resolution order for the reasoning task from memory?
- Can I name three things I'd improve and why?
- Have I actually run /api/v1/rag/query and /api/v1/advisor/shipping in the last 24 hours?

12. Code Tracing Maps

Each map below traces a real request through every file it touches, in order. Open them side-by-side with the codebase and you can read along line by line. These are the maps to rehearse before a whiteboard interview.

12.1 Quote Flow (Java API)

```
Frontend (apps/web)
  └─ POST /api/v1/quotes  { origin, destination, package, ... }
        │
        ▼
apps/api-java
  ├─ controller/QuoteController.getQuote()
  │     - @Valid QuoteRequest dto         (Bean Validation runs here)
  │     - @PreAuthorize / SecurityFilter  (JWT extracted upstream)
  │
  ├─ service/QuoteService.computeQuotes()
  │     - business rules, normalisation
  │     - calls carrier comparison logic
  │     - @Transactional boundary opens
  │
  ├─ repository/QuoteRepository  (Spring Data JPA)
  │     - persists the quote request for audit
  │
  └─ returns QuoteResponse
        - list of carrier options
        - validation errors → 400 via @ControllerAdvice
        - auth failures     → 401/403 via SecurityFilterChain
        │
        ▼
Frontend renders the comparison table.
```

12.2 Advisor Flow (Python API)

```
Frontend AdvisorPage
  └─ POST /api/v1/advisor/shipping  { query, context }
        │
        ▼
apps/api-python/app/api/routes/advisor.py:32  shipping_advisor()
  - rag           = request.app.state.rag
  - tool_registry = request.app.state.tool_registry
  - llm_router    = request.app.state.llm_router
  - reasoning_client = llm_router.for_task(TASK_REASONING)   # → OpenAI
        │
        ▼
app/services/shipping_advisor_service.py  get_shipping_advice()
  ├─ embedding_provider.embed([query])
  ├─ vector_store.search(qvec, top_k=settings.rag_top_k)     # cosine
  ├─ if context has address dict:
  │     tool_registry.get("validate_address").execute(...)
```

```
  │     tool_registry.get("get_quote_preview").execute(...)
  ├─ build_advisor_prompt(query, retrieved_chunks, tool_results)
  └─ reasoning_client.complete(messages)                    # OpenAI
        │
        ▼
ShippingAdvisorResponse {
  answer, reasoning_summary, tools_used,
  sources [{source, chunk_index, score}],
  context_used
}
```

12.3 Recommendation Flow (Python API)

```
Frontend RecommendationCard (after a quote is fetched)
  └─ POST /api/v1/advisor/recommendation  { services, context }
        │
        ▼
app/api/routes/advisor.py:106  get_recommendation()
  - llm_router = request.app.state.llm_router
  - llm_client = llm_router.for_task(TASK_SYNTHESIS)         # → Gemini
        │
        ▼
app/services/recommendation_service.py  generate_recommendations()
  ├─ score each service deterministically
  │     (price, speed, reliability — pure code, no LLM)
  ├─ classify primary vs alternatives by score
  ├─ if llm_client present:
  │     summary = await _generate_summary(primary, alts, llm_client)
  │     (this is the ONLY place the LLM touches recommendations)
  └─ return RecommendationResult
        │
        ▼
RecommendationResponse {
  primary_recommendation, alternatives, summary, metadata
}
```

Important detail: scoring is deterministic. The LLM only writes the human-readable summary. If the LLM is unavailable we still return ranked options — the user-facing failure mode is 'no nice paragraph', not 'no recommendation'.

13. Failure Modes & Fallback Behaviour

For each failure mode: what triggers it, what the system actually does, what the user sees, and what to verify.

Failure	Trigger	System behaviour	User-visible result
Missing OpenAI key	LLM_PROVIDER_REASONING=openai but OPENAI_API_KEY empty	build_provider_client returns None → router falls through to LLM_PROVIDER_FALLBACK (echo). Warning logged: 'Provider openai requested but OPENAI_API_KEY is not set'.	Advisor still answers but with EchoClient — returns the retrieved context verbatim with a note. Synthesis (Gemini) unaffected.
Missing Gemini key	LLM_PROVIDER_SYNTHESIS=gemini but GEMINI_API_KEY empty	Same path as above for the synthesis task only. Reasoning (OpenAI) unaffected.	/rag/query and recommendation summary use Echo. /advisor/shipping still uses OpenAI.
Both LLM keys missing	LLM_PROVIDER set but no keys, or LLM_PROVIDER unset entirely	Both tasks degrade to LLM_PROVIDER_FALLBACK (echo). App boots normally — never crashes.	All AI endpoints return retrieved-context responses. Frontend works; just no AI prose.
Missing carrier credentials	SHIPPING_PROVIDER=ups with empty UPS_CLIENT_ID/SECRET	_has_required_credentials() returns False → factory falls back to MockShippingProvider. Warning logged.	Quote previews and address validation return

			mock data. Tools still work end-to-end.
Empty vector store	Service started but /rag/ingest never called, or data/documents empty	vector_store.search() returns []. RAG service passes empty context list to build_rag_prompt, which inserts 'No context was retrieved'. LLM is told to say it doesn't know.	Advisor answers truthfully with 'no informatio n available'. No hallucinate d answer.
Python API down	FastAPI process crashed, port unreachable	Frontend fetch fails. Java API is unaffected — quoting and saved options still work.	Advisor card shows error state. Quote form continues working normally.
RAG returns no matching chunks	Query is unrelated to any document	Top-K still returns the K closest vectors but with low scores. Prompt includes them; system prompt instructs grounding.	Model typically responds 'the provided context does not cover this'. Sources panel shows the low-score chunks.
LLM provider call raises mid-request	Network error, 5xx, rate limit	OpenAIClient/GeminiClient catch and raise AppError(502). Global error handler returns clean JSON.	Frontend sees a 502 with a message. Other endpoints unaffected.
Tool execution fails	Provider returned an error or input failed validate_input	AppError raised; route returns it. tools_used reflects what ran.	Advisor returns the LLM answer derived

			from RAG context only, minus the failed tool's data.
Bad LLM_PROVIDER_FALLBACK name	LLM_PROVIDER_FALLBACK=not-a-real-provider	build_provider_client returns None; router catches and uses EchoClient. Tested in test_router_never_crashes_on_bad_fallback_name.	Same as Echo fallback. App still boots.

Design principle behind every row above: degrade in place, never crash on startup, always log the reason. The app boots with an entirely empty .env.

14. Spring Boot Backend — Deeper Dive

14.1 Request Lifecycle (one quote round-trip)

```
HTTP POST /api/v1/quotes
        │
        ▼
┌──────────────────────────────────────┐
│ Tomcat / Embedded Servlet Container  │
└──────────────────────────────────────┘
        │
        ▼
┌──────────────────────────────────────┐
│ Spring SecurityFilterChain           │
│  - JwtAuthenticationFilter           │
│  - extracts Authorization: Bearer ...│
│  - validates against Supabase JWKS   │
│  - sets SecurityContextHolder        │
└──────────────────────────────────────┘
        │
        ▼
┌──────────────────────────────────────┐
│ DispatcherServlet → @RestController  │
│  QuoteController.getQuote(@Valid dto)│
│   - Bean Validation runs on dto      │
│   - @PreAuthorize checks role/scope  │
└──────────────────────────────────────┘
        │
        ▼
┌──────────────────────────────────────┐
│ @Service QuoteService                │
│  - business rules                    │
│  - @Transactional opens TX           │
│  - calls repository                  │
└──────────────────────────────────────┘
        │
        ▼
┌──────────────────────────────────────┐
│ Spring Data JPA Repository           │
│  - generated SQL via Hibernate       │
│  - persists audit row                │
└──────────────────────────────────────┘
        │
        ▼
┌──────────────────────────────────────┐
│ TX commits → DTO mapper → JSON       │
│ HTTP 200 + QuoteResponse             │
└──────────────────────────────────────┘

Errors:
  Validation  → @ControllerAdvice → 400 + field errors
  Auth        → SecurityFilterChain → 401/403
```

```
Not found   → custom exception → 404
Anything else → fallback handler → 500 (logged)
```

14.2 DTO Validation

Inputs are Java records / classes with Bean Validation annotations (@NotNull, @Positive, @Size, @Pattern). The controller method parameter is annotated @Valid, which makes Spring run the validators before the controller body executes. Validation failures throw MethodArgumentNotValidException, which a @ControllerAdvice maps to a 400 with a structured field-error list. The service layer never sees an invalid DTO.

14.3 Service Layer

Services hold the business rules. They are stateless beans, constructor-injected with their dependencies (repositories, other services). Why a layer at all instead of putting logic in controllers? Three reasons:

- Controllers stay HTTP-shaped (status codes, headers); services stay domain-shaped. Different testing strategies.
- @Transactional belongs at the service boundary, not the controller, because one HTTP request might invoke several services.
- Reuse: a scheduled job or a background worker can call the same service without going through HTTP.

14.4 Repository / Persistence

Repositories extend JpaRepository<Entity, Id>. Spring Data generates implementations from method names (findByUserIdOrderByCreatedAtDesc) and gives you crudby and page-by-x for free. Hibernate is the JPA provider; the actual DB is Supabase Postgres. Entities are mapped with @Entity / @Id / @Column. The repository layer is the only place that talks SQL — everything above thinks in domain objects.

14.5 JWT / Auth Flow

```
Frontend
  - User signs in via Supabase Auth (handled in the SPA)
  - Receives a signed JWT (RS256)
  - Sends Authorization: Bearer <jwt> on every API call

Java API
  - SecurityFilterChain registers JwtAuthenticationFilter
  - Filter calls JwtDecoder.decode(token)
      (decoder fetches Supabase JWKS; verifies signature + exp + iss)
  - Builds an Authentication with sub/email/roles claims
  - Stores it in SecurityContextHolder
  - Controller methods can use @PreAuthorize and @AuthenticationPrincipal
  - Saved-options + bookings are scoped to the authenticated user id
```

14.6 Transaction Boundaries

@Transactional is applied at the service method level. Default propagation is REQUIRED — if a transaction is already open it joins, otherwise it starts one. Read-only methods are annotated @Transactional(readOnly = true) so Hibernate skips dirty checking. Anything that mutates more than one row (saving a quote + writing an audit row) lives in one method so the whole thing is atomic. Crucially: never put @Transactional on the controller — controllers may call multiple services, and you want each to manage its own boundary.

15. Backend Design Patterns Used Here

Naming the patterns out loud is a senior-interview move. Each one below is paired with the exact file in this repo where it lives, so the answer is never abstract.

15.1 Factory

create_llm_client(), create_llm_router(), create_shipping_provider(), create_embedding_provider(), create_vector_store(). Each one reads config and returns the right concrete implementation. Callers depend on the ABC, not the constructor of any specific class. Adding a new provider = one branch in one factory.

15.2 Adapter

UPSProvider, FedExProvider, DHLProvider, USPSProvider all adapt their carrier's HTTP/JSON shape into the project's ShippingProvider interface (validate_address, get_quote_preview). The rest of the system never sees a carrier-specific field name. GeminiClient is also an adapter — it converts OpenAI-style {role, content} messages into Gemini's contents/parts format inside _messages_to_gemini_contents.

15.3 Strategy

LLMRouter is strategy with a twist: instead of one strategy per request, it picks one strategy per task at startup and reuses it. The 'algorithm' you're swapping is which LLM client handles which logical task. Same shape — interface + interchangeable implementations selected by config.

15.4 Graceful Degradation

Every factory has a fallback path. Every external call has a safe default. Examples already cited: bad creds → mock provider; missing key → echo; bad fallback name → echo; empty vector store → 'no context' branch in the prompt. The system can boot with an empty .env and serve requests.

15.5 Composition Root

main.py's lifespan() function is the composition root. It is the only place that constructs concrete classes and wires them together. Every other file works against ABCs and receives its dependencies. This makes tests trivial: the test just builds its own app.state with fakes.

15.6 Abstraction Boundaries

- Routes know about HTTP and Pydantic schemas, not LLMs.
- Services know about LLMClient and ToolRegistry, not OpenAI/Gemini specifics.
- Clients know about one provider each, not about routing or tasks.
- The router knows about tasks and clients, not about HTTP or RAG.

Each layer has exactly one reason to change. That is the single-responsibility principle in practice, and it is what makes this codebase explainable in 60 seconds.

16. Design Tradeoffs

For each decision: what we picked, the alternative we rejected, and the cost we accepted by picking it. Senior interviewers care more about the tradeoff than the choice.

16.1 Why the Frontend Orchestrates Two Backends

- Picked: frontend calls Java and Python directly, merges results in the UI.
- Alternative: a BFF / API gateway sits in front, fanning out to both.
- Cost accepted: the frontend has to know two base URLs and two error shapes. We do not have a single auth/rate-limit chokepoint.
- Reason: it removes one deployable, eliminates an internal RPC channel that could fail, and makes failure modes obvious to the user (one card errors, the rest of the page works).

16.2 Why Java and Python Are Split

- Picked: Spring Boot for transactions; FastAPI for AI.
- Alternative: one polyglot service (Java with a Python sidecar) or one all-Python service.
- Cost accepted: two CI pipelines, two language ecosystems, two deploys.
- Reason: each service can use its strongest ecosystem (Spring for transactional, Python for LLM/embedding libs) and a flaky AI dependency cannot bring down the booking flow.

16.3 Why Deterministic Scoring Before LLM Explanation

- Picked: rank options in pure code, then ask the LLM to write a paragraph about the top one.
- Alternative: ask the LLM to do the ranking too.
- Cost accepted: the prose is constrained to 'explain this winner' instead of 'choose freely'.
- Reason: ranking is the part the user can verify against price/ETA columns. Hallucinated rankings would erode trust immediately. LLMs are great writers and bad spreadsheets — give them the writing job.

16.4 Why an In-Memory Vector Store First

- Picked: InMemoryVectorStore behind a VectorStore ABC.
- Alternative: pgvector / Qdrant / FAISS from day one.
- Cost accepted: data is lost on restart; ingestion is a route, not a job; performance ceiling around ~10k chunks.
- Reason: the bottleneck of a RAG system is rarely the index at this scale. Putting an ABC in place means the swap is one factory edit when the corpus grows. We chose to demonstrate the architecture, not over-engineer the index.

16.5 Why Task-Based LLM Routing

- Picked: a router that maps logical tasks (reasoning, synthesis, fallback) to clients, configured per env var.
- Alternative: one global LLM client; or per-request dynamic selection with scoring.
- Cost accepted: two SDK surfaces, two failure modes, slight tone variation between tasks.

- Reason: cost (Gemini Flash for synthesis is ~10× cheaper), failure isolation, and operability — ops change a single env var to swap a task's provider without redeploying code.

16.6 Why Tools Instead of Pure LLM Reasoning

- Picked: deterministic tools for ground-truth questions; LLM only for advice/synthesis.
- Alternative: ask the LLM to compute everything from training-time knowledge.
- Cost accepted: more code (Tool ABC, registry, schemas) and more handoffs.
- Reason: LLMs hallucinate facts. ZIP validation and quote pricing are exactly the questions where 'sounds right' is not good enough. Tools make those answers boring and correct.

17. Debugging & Validation

How to manually convince yourself each subsystem is working. Run these in order top to bottom — each step proves the next one's prerequisites.

17.1 Manual Test Sequence

28. Boot the Python API with an empty .env. Expected log: 'LLM router initialized: {reasoning: echo, synthesis: echo, fallback: echo}'. Proves: factories degrade safely.
29. GET /health → 200. Proves: process is up, routes mounted.
30. GET /api/v1/orchestration/tools → list of 2 tool schemas. Proves: tool registry built.
31. POST /api/v1/rag/ingest → response with chunks_ingested > 0. Proves: documents found, chunked, embedded, stored.
32. POST /api/v1/rag/query with a known-content query → answer + sources array non-empty. Proves: retrieval + prompt + LLM (or Echo) path.
33. POST /api/v1/advisor/shipping with a real query → answer + tools_used + sources. Proves: reasoning task wired, services compose retrieval + tools + LLM.
34. POST /api/v1/advisor/recommendation with sample services → primary_recommendation + alternatives + summary. Proves: deterministic scoring runs even when LLM is Echo.
35. Set OPENAI_API_KEY and LLM_PROVIDER_REASONING=openai, restart. Re-run /advisor/shipping. Log should now show provider=openai. Proves: live LLM path.
36. Unset OPENAI_API_KEY but keep LLM_PROVIDER_REASONING=openai. Restart. Expected warning + reasoning falls to fallback. Proves: per-task degradation.

17.2 Logs to Inspect

- Startup: 'Starting shipsmart-api-python ...', 'LLM router initialized: {...}', 'Tool registry initialized: 2 tools, provider=mock'. If any of these are missing, that subsystem failed to wire.
- Per-task resolution: 'LLM router: task=reasoning → provider=openai'. Proves which provider each task picked.
- Warnings: 'Provider openai requested but OPENAI_API_KEY is not set' or 'Failed to create provider ups: ... — falling back to mock'. These are the smoking guns for misconfig.
- Per-request: RequestLoggingMiddleware logs the method, path, status, and duration. Useful for spotting slow LLM calls.
- Errors: AppError-derived exceptions are formatted by the global handler; 502s typically mean an LLM provider returned an error.

17.3 What 'Working' Looks Like Per Subsystem

Subsystem	How to exercise it	Output that proves it works
Config	boot with .env	No exceptions; defaults applied for unset vars.
Lifespan / wiring	boot logs	All five 'initialized' lines present.
Tool registry	GET /api/v1/orchestration/tools	JSON list with validate_address +

		get_quote_preview schemas.
Shipping provider	GET /api/v1/orchestration/tools or run a tool	Provider name in startup log; tool execution returns sane data.
Embeddings	POST /rag/ingest	chunks_ingested > 0; no zip(strict=True) error.
Vector store	POST /rag/query	sources array non-empty for an in-corpus query.
LLM router	startup logs	'LLM router initialized: {reasoning, synthesis, fallback}' line.
Reasoning task	/advisor/shipping with key set	Coherent answer; not the Echo template.
Synthesis task	/rag/query with key set	Coherent answer; not the Echo template.
Per-task fallback	unset one key, restart, hit that endpoint	Echo-style answer for that task only; the other still uses its real LLM.
Frontend integration	AdvisorPage in browser	Card renders sources and answer; loading + error states correct.

18. Interview Practice Pack

Each block: question — strong sample answer in your own voice. Read them out loud at least twice. The point is not to memorise the wording, it is to internalise the structure: decision → tradeoff → evidence in code.

18.1 Senior Backend Questions

Q. Walk me through how a single HTTP request becomes a database row.

On the Java side: the request hits Tomcat, then the Spring SecurityFilterChain validates the Supabase JWT and populates SecurityContextHolder. DispatcherServlet routes to the @RestController method, which receives a @Valid DTO — Bean Validation runs before the body executes. The controller calls a @Service method, which is where @Transactional opens the transaction. The service uses a Spring Data JPA repository to persist via Hibernate, the transaction commits, the response DTO is mapped, and Spring serialises it to JSON. Failures route through @ControllerAdvice into structured error responses.

Q. How do you handle errors and validation consistently?

Validation lives at the boundary using Bean Validation annotations on DTOs, triggered by @Valid. A @ControllerAdvice maps validation exceptions to 400s with field-level details. Domain errors throw typed exceptions that the same advice maps to 4xx codes. Anything unexpected falls through to a 500 handler that logs the stack trace and returns a sanitised body. On the Python side the equivalent is AppError + register_error_handlers in app/core/errors.py.

Q. Why do you put @Transactional on the service and not the controller?

Because the controller is HTTP-shaped and might call several services. Each service should own its own transactional boundary. Putting @Transactional on the controller means a transaction stays open across multiple service calls, which blurs ownership and makes rollback semantics fuzzy. It also couples HTTP concerns to persistence concerns.

Q. How would you scale this backend?

Java side: it is already stateless, so horizontal scaling behind a load balancer is the first move. Push read-heavy queries through a read replica. Add Redis for hot data. Python side: same — stateless apart from app.state caches that are easy to externalise. The only shared state today is the in-memory vector store, and that is behind an ABC, so swapping to pgvector is a one-factory edit. The harder scaling problem is LLM cost — task-based routing already addresses that by sending the cheaper task to a cheaper provider.

18.2 RAG Questions

Q. Walk me through your RAG pipeline.

On startup the service walks data/documents recursively, chunks each .md/.txt into ~500-character overlapping windows, embeds them — text-embedding-3-small in production, a deterministic hash embedding in dev — and stores them in an in-memory vector store behind

an ABC. At query time we embed the query, take cosine top-3, and call build_rag_prompt() which puts the chunks in a system-grounded prompt that tells the LLM to answer only from the provided context. The endpoint returns the answer plus the sources (file path, chunk index, score) so the UI can cite them.

Q. How do you avoid hallucination?

Five layers. First, retrieval: we put real document text in the prompt so the model has something to ground on. Second, system prompt: it explicitly says answer only from the provided context. Third, tools: ground-truth questions (addresses, quotes) go through deterministic tools, never the LLM. Fourth, citations: every answer comes with sources, so the user can verify. Fifth, fallback: if the LLM is unavailable we return retrieved context verbatim via EchoClient instead of an empty hallucination.

Q. Why chunking with overlap? Why not embed full documents?

Two reasons. One, the LLM has a context limit; you can't stuff a whole policy doc into every prompt. Two, embeddings average semantics across the input — a long document produces a vector that represents nothing in particular. Chunking gives each section its own vector, so the retrieval step can rank passages independently. The overlap exists so a sentence that straddles a chunk boundary still appears intact in at least one chunk.

Q. What would change at 10 million documents?

Index, ingestion, and embedding costs. The InMemoryVectorStore would be replaced with pgvector or Qdrant — same ABC, swap the factory. Ingestion would move out of the request path into a background job. We'd batch and cache embedding calls. We'd add metadata filters (carrier, region) so retrieval narrows the candidate set before ranking. None of this requires touching the services or routes — that's the whole point of the abstraction.

18.3 MCP / Tooling Questions

Q. Why do you have a tool layer at all? Why not let the LLM answer?

Because some questions have a right answer and the LLM doesn't know it. 'Is this a valid US ZIP code?' and 'how much will UPS charge for this package?' should be computed by code that talks to a real API, not predicted from training data. The tool layer is where ground truth lives. The LLM stays in the lane it is good at — explaining the answer in natural language.

Q. How is your tool layer like MCP?

Same shape. Each tool exposes name, description, parameters with types, and an execute method. The registry lists schemas via /api/v1/orchestration/tools — that is the discovery surface. Inputs are validated before execution. The handoff between text reasoning and structured execution is what MCP standardises, and we model it the same way internally.

Q. How do you keep tool outputs and LLM reasoning consistent?

Tool outputs are appended to the LLM prompt as evidence before the model writes its answer. The system prompt instructs the model to use the tool data as ground truth. The response back

to the client includes both the LLM text and a tools_used list, so the UI can show what was computed vs. what was reasoned. If the user sees a price in the answer, they also see which tool produced it.

19. Must Know vs Nice to Know

If you only have an hour, study column 1. Column 2 is depth that turns a good answer into a great one. Column 3 is genuinely optional and only matters if the interviewer drills in.

MUST KNOW	Nice to know	Optional / depth
3-service architecture; Java for transactions, Python for AI; frontend orchestrates	Why the two backends never call each other (failure isolation, independent deploy)	Comparison with BFF / API gateway alternatives
RAG = retrieve, then ground the LLM in the retrieved text	Chunking with overlap; cosine top-K; sources returned for citation	Hybrid retrieval (BM25 + dense), reranking, eval metrics like recall@k
Tools exist to handle ground-truth questions instead of letting the LLM guess	Tool ABC + registry + schemas; how it mirrors MCP	Function calling vs explicit tool dispatch; tool authorisation
Task-based LLM router: reasoning → OpenAI, synthesis → Gemini, fallback → Echo	Per-task resolution order; per-task independent fallback; cost rationale	Per-task tuning (timeout/temperature), provider-side retries, A/B routing
Composition root in main.py lifespan(); everything else uses ABCs	Why building once at startup matters (cost, testability)	Dependency injection containers vs hand-wired roots
Spring Boot request lifecycle: filter chain → controller → service → repo	@Transactional on the service layer, not the controller	Propagation modes, isolation levels, optimistic locking
DTO validation with @Valid + Bean Validation; @ControllerAdvice for errors	Why validation must live at the boundary	Custom validators, group validation, cross-field constraints
JWT auth from Supabase, validated in Spring SecurityFilterChain	JWKS rotation, claims-based authorization with @PreAuthorize	Token refresh strategies, clock skew, audience claims
Graceful degradation: every factory has a safe default	Specific fallbacks: bad creds → mock; missing key → echo	Circuit breakers, bulkheads, exponential backoff per provider
Deterministic scoring before LLM explanation in recommendations	Why ranking is in code and only the prose is in the LLM	Learning-to-rank, ranking models, score calibration

Strategy for the day before: re-read column 1 once, then do every Out-Loud item in §10. If anything in column 1 is shaky, stop studying new material and rehearse it.